Einfache C-Programme

Grundsymbole

- **Bezeichner**
  - Variablen, Typen, Funktionen
  - bestehen aus Buchstaben, Ziffern, Sonderzeichen '_'
  - beginnen mit einem Buchstaben oder ein '_'
  - Groß-/Kleinschreibung signifikant

```
x
x17
  x17
my_Var
my_var
```

- **Reservierte Wörter**
  - haben eine in der Sprache festgelegte Bedeutung
  - dürfen nicht als Bezeichner verwendet werden

```
if
while
```

- **Zahlen**
  - ganze Zahlen (dezimal, hexadezimal, oktal)
  - Gleitkommazahlen

```
376       dezimal
0x1A5     hexadezimal
042       oktal
3.14      Gleitkommazahl
```

- **Zeichen und Zeichenketten**
  - zwischen einzelnen / doppelten Hochkommata
  - dürfen nicht über mehrere Zeilen gehen

```
'A'
"a simple string"
"sie sagte \" Hallo \""
```

- **Operatoren**
  - arithmetisch, relational, logisch

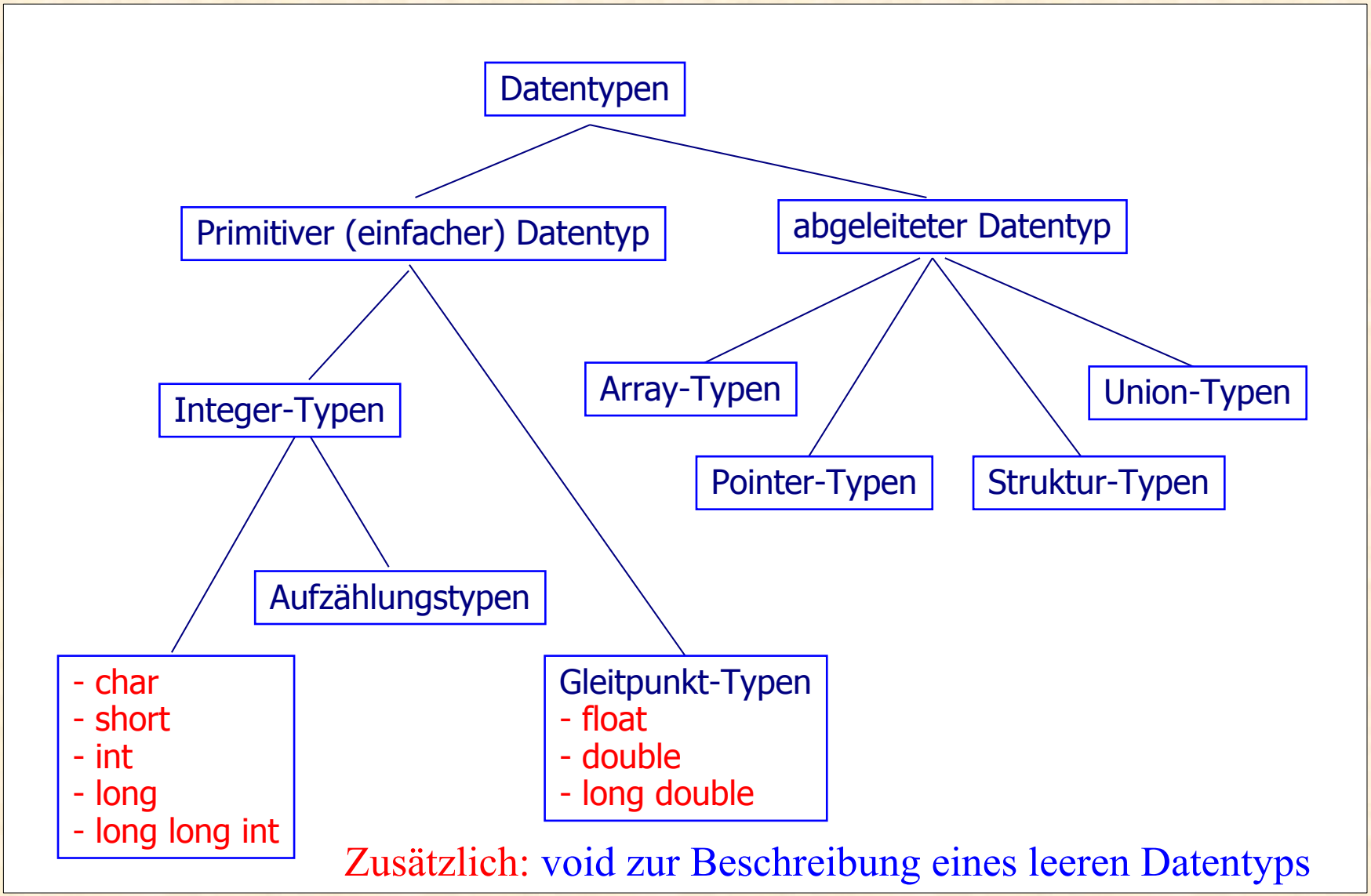
Reservierte Wörter

auto	do	for	return	switch	while
break	double	goto	short	typedef	
case	else	if	signed	union	
char	enum	int	sizeof	unsigned	
continue	extern	long	static	void	
default	float	register	struct	volatile	

der C99 bzw. C11-Standard verfügt über weitere Schlüsselwörter:

inline	_Alignas	_Atomic	_Complex	_Imaginary	_Static_assert
restrict	_Alignof	_Bool	_Generic	_Noreturn	_Thread_local

Datentypen in C



Standard-Integer-Typen von C90

Datentyp	Anzahl Bytes	Wertebereich (dezimal)
char	1	-128 bis 127 oder 0 bis +255
unsigned char	1	0 bis 255
signed char	1	-128 bis +127
unsigned short int	2 in der Regel	0 bis +65535
short int	2 in der Regel	-32768 bis +32767
int	4 in der Regel	-2147483648 bis +2147483647
unsigned int	4 in der Regel	0 bis +4294967295
long int	4 in der Regel	-2147483648 bis +2147483647
unsigned long int	4 in der Regel	0 bis +4294967295

Für die Implementierung der ganzzahligen Datentypen gilt:

- short und int müssen wenigsten 16 Bits haben, long mindestens 32 Bits
- long $\supseteq$ int $\supseteq$ short $\supseteq$ char
- verwendete Wertebereichsgrenzen in <limits.h> bzw. <float.h> definiert

Aber: mit C99-Standard wurden ganzzahlige Datentypen fester Größer eingeführt

- wie int8_t, int16_t, int32_t, uint8_t, uint16_t und uint32_t in <stdint.h>

Ganzzahlige Datentypen int, short und long

- Begrenzter Wertebereich: Überlauf/Unterlauf möglich
- Modulo-Arithmetik: falsche Werte
- Überlaufbit gesetzt: errno (globale Variable)

- ganze Zahlen werden im **Zweierkomplement** abgespeichert

Beispiel: Umrechnung von – 4 ins Zweierkomplement

Vorzeichen ignorieren: 4(10) = 00000100(2)

Invertieren: Not[00000100] = 11111011

Eins addieren: 11111011 + 00000001 = 11111100

-4(10) = 11111100(2)

- **Aber warum macht man das so kompliziert?**

Ganzzahlige Datentypen int, short und long

- zusätzlicher Datentyp in C11: (unsigned) long long int

Datentyp	Anzahl Bytes	Wertebereich (dezimal)
long long int	8 in der Regel	-9223372036854755808 bis +9223372036854755807
unsigned long long int	8 in der Regel	0 bis +18446744073709551615

Variablendeklaration

- jede Variable muss vor ihrer Verwendung deklariert werden
  - macht den Namen und den Typ der Variablen bekannt
  - Compiler reserviert Speicherplatz für die Variable

```
int x;          deklariert eine Variable x vom Typ int
short a,b;      deklariert 2 Variablen a und b vom Typ short (short int)
```

- Initialisierungen

```
int x=100;          deklariert eine int-Variable x mit Anfangswert 100
short a=1, b=2;     deklariert 2 short-Variablen a und b mit Anfangswerten
```

- C unterscheidet zwischen lokalen und globalen Variablen
  - globale Variablen sind im gesamten Programm, lokale nur innerhalb einer Funktion, sichtbar

Konstantendeklaration

Initialisierte „Variablen“, deren Wert man nicht mehr ändern kann

- durch Präprozessor-Anweisung (am Dateianfang) oder Schlüsselwort const definierbar

```
const  int MAX = 100;
```

oder

```
/* Datei: beispiel.c */
        .....
#define MAX 100
        .....
int main(void){
        .....
}
```

Zweck

- bessere ***Lesbarkeit***

  MAX ist besser lesbar als 100

- bessere ***Wartbarkeit***

  kommt die Konstante mehrmals vor und muss geändert werden, dann muss das nur an einer einzigen Stelle erfolgen

Kommentare

Geben Erläuterungen zum Programm

- Einzeilige Kommentare

```
// Eine einzelne Kommentarzeile
// Erfunden für C++
```

- Mehrzeilige Kommentare

```
/* Ein mehrzeiliger Kommentar.
   Kommentare können nicht geschachtelt werden */
```

Verwendung von Bezeichnern

- grundsätzlich sollen ***sprechende Bezeichner*** verwendet

  Beispiel: **arrayOutOfBoundError**

- Ausnahme: Hilfsvariablen die man nur über kurze Strecken benutzt

  wie: i und j

- Konstanten werden vollständig groß geschrieben

  const int MAX = 100;

- Bezeichner für Variablen, Funktionen, ... werden klein geschrieben

  int sum;

Verwendung von Bezeichnern

- setzen sich Namen aus mehreren Worten zusammen, gibt es verschiedene Konventionen:

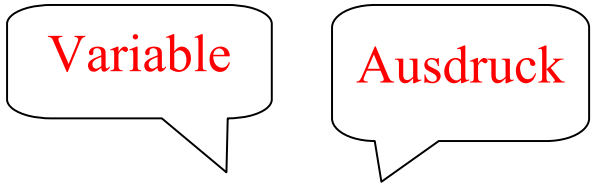
```
int getPreferredSize(...);        // “camel case“
int get_preferred_size(...);      // „snake case“
```

Grundsätzlich gibt es in C keine verbindlichen Namens- und Formatierungskonventionen

Es gibt aber Projekt- oder Firmen-spezifischen Konventionen

Ein einheilicher Look-and-Feel von Programmen ist sehr wichtig für die Lesbarbeit von Quell-Code.

Zuweisungen in C



1. Berechne den Ausdruck
2. speichere seinen Wert in der Variablen

```
x = y + 1;
```

Zuweisungsregel in C: sind für eine Zuweisung, die Datentypen der rechten und linken Seite ungleich, aber verträglich, dann wird der Wert des Ausdrucks implizit in den Typ der Variablen umgewandelt und der Variablen zugewiesen

- arithmetische Typen sind in C verträgliche Typen

Beispiele:

```
int i,j; short s; char b;
i  = 300;  // ok: Zahlenkonstanten sind int
i  = j;    // ok: derselbe Typ
b = 300;   // ok, aber Warnung:
           // 300 passt nicht in char
i = s;     // ok
s = i;     // ok, aber keine Warnung, obwohl
           // fehlerhafte "Rundung" möglich ist
```

Vorsicht: implizite Typumwandlungen sind gefährlich, da sie oft nicht richtig eingeschätzt werden und zu Informationsverlusten führen können

Einschub: Zuweisungen in Java und C#

In stark typisierten Programmiersprachen - wie Java und C# - dürfen Zuweisungen nur verwendet werden, wenn sie zu keinem Informationsverlust führen

Bedingung: linke und rechte Seite müssen ***zuweisungskompatibel*** sein

- müssen denselben Typ haben,
  oder $\text{Typ}_{\text{links}} \supseteq \text{Typ}_{\text{rechts}}$

Hierarchie der ganzzahligen Typen:

long ⊇ int ⊇ short ⊇ byte

Beispiele:

```
int i,j; short s; byte b;
i  = 300;    // ok: Zahlenkonstanten sind int
i  = j;      // ok:derselbe Typ
b = 300;     // Fehler!: 300 passt nicht in byte
i = s;       // ok
s = i;       // Fehler!
```

Statische Typenprüfung: Compiler überprüft,

- dass Variablen nur erlaubte Werte enthalten
- dass auf Werten nur erlaubte Operationen ausgeführt werden

Arithmetische Ausdrücke

- **Vereinfachte Grammatik**

Expr = Operand [BinaryOperator Operand]
Operand = [UnaryOperator] (Identifier | number | "(" Expr")")

- **Binäre Operatoren**

\+ Addition
\- Subtraktion
* Multiplikation
/ ganzzahlige Division 4/3=1 (-4)/3=-1 4/(-3)=-1 (-4)/(-3)=1
% Modulo (Rest der Division)
4%3=1 (-4)%3=-1 4%(-3)=-1 (-4)%(-3)=1

- **Unäre Operatoren**

\+ Identität +x
\- Vorzeichenumkehrung -x

Vorrang- und Typregeln in arithmetischen Ausdrücken

- **Vorrangregeln (Präzedenzregeln):**

  - Punktrechnung (*, /, %) vor Strichrechnung (+, -)
  - Operatoren auf gleicher Stufe werden entsprechend ihrer Assoziativität (von links nach rechts oder umgekehrt) verknüpft
  - aber: Reihenfolge der Auswertung der Operanden ist undefiniert
  - unäre Operatoren binden stärker als binäre

  z.B. 3 + 4 * -2 ergibt -5

  a = f(par++, par++) ist undefiniert

- **Typregeln:** für arithmetische Operanden gilt, der "kleinere" Datentyp wird in den "größeren" Datentyp umgewandelt

Beispiel:

Ausdruck	Ergebnistyp
2 * 3L + 1.1	double

Präzedenzregeln: Vollständiger Überblick

Priorität	Operator	Assoziativität
15	( ) [ ] -> .	von links nach rechts
14	! ~ ++ -- + - (TYP) * & sizeof	von rechts nach links
13	* / % (Rechenoperationen)	von links nach rechts
12	+ - (binär)	von links nach rechts
11	<< >>	von links nach rechts
10	< <= > >=	von links nach rechts
9	== !=	von links nach rechts
8	&	von links nach rechts
7	^	von links nach rechts
6	|	von links nach rechts
5	&&	von links nach rechts
4	||	von links nach rechts
3	?:	von rechts nach links
2	= += -= /= *= %= >>= <<= &= |= ^=	von rechts nach links
1	, (Sequenz-Operator)	von links nach rechts

Später mehr!



Ganzzahltypen: Typkonvertierungen

- ***Automatische Typkonvertierung*** erfolgt
  - bei einer Zuweisung x = Ausdruck, wenn der Typ der Variablen und des zugewiesenen Ausdrucks nicht identisch sind

    Ausdruck ⟶ Typ von x
  - beim Aufruf einer Methode, falls die Typen der aktuellen Parameter nicht mit denen der formalen Parameter übereinstimmen

    aktuelle Parameter ⟶ formale Parametertypen
  - bei einer Auswertung eines arithmetischen Ausdrucks, wenn die Operanden unterschiedlich sind

    char / short ⟶ int / unsigned int ⟶ long / unsigned long
- ***Explizite Typkonvertierung***

  Allgemeine Form: (typename) value

  Beispiel: int x,y;

  ....

  (short) x/y;

Einschub: Typkonvertierungen in Java und C#

- Automatische Typkonvertierung (widening conversion) erfolgt in stark typisierten Sprachen - wie Java oder C# - nur dann wenn keine Information verloren geht
  - Im Allgemeinen: wenn der Wert eines kleineren Typs in dem eines größeren Typs überführt werden soll

byte ⟶ short ⟶ int ⟶ long

- Aber: bei expliziter Typkonvertierung (narrowing conversion) kann auch in streng typisierten Sprachen zu Informationsverlust kommen
  - man nimmt an, dieses ist vom Programmierer gewollt

Inkrement- und Dekrement-Operatoren

- Variablenzugriff kombiniert mit Addition/Subtraktion

x++	nimmt den Wert von x und erhöht x anschließend um 1
++x	erhöht x um 1 und nimmt anschließend den Wert von x
x--	nimmt den Wert von x und verringert x anschließend um 1
--x	verringert x um 1 und nimmt anschließend den Wert von x

Beispiele

```
x = 1; y = x++ * 3;      // x = 2, y = 3     entspricht: y = x * 3; x = x + 1;
x = 1; y = ++x * 3;      // x = 2, y =6      entspricht: x = x + 1; y = x * 3;
```

- Kann auch als eigenständige Anweisung verwendet werden

```
x = 1; x++;              // x = 2            entspricht x = x+1;
```

- darf nur auf Variablen angewendet werden, nicht auf Ausdrücke

```
y = (x+1)++;             // falsch
```

Zuweisungsoperatoren

- Zuweisung ist in C selbst als Operator definiert und kann deshalb auch Teil eines Ausdrucks sein

```
y = 1 + (x = 1 + 2) + 3      // y = 7, x = 3      entspricht x = 1+2; y = 1 + x + 3
```

Spart Schreibarbeit, ist aber nicht schneller als die Langform

Lassen Sie den Quatsch, der Code wird dadurch schlecht lesbar

Gerade noch erlaubt:

```
x = y = 1;    // Der Zuweisungsoperator ist rechtsassoziativ.
```

Zuweisungsoperatoren

- Arithmetische Operatoren lassen sich mit Zuweisung kombinieren

	Kurzform	Langform
+=	x += y	x = x + y
-=	x -= y	x = x – y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y

spart Schreibarbeit, ist aber nicht schneller als die Langform

Aufzählungstypen

- Aufzählungstypen sind nicht vordefinierte skalare Typen, die oft zur Beschreibung von Mengen verwendet werden
- Definition von Aufzählungstypen

```
enum Color { RED, GREEN, BLUE, YELLOW };
enum Color c = RED;
```

- Datentyp als auch die Aufzählungskonstanten werden definiert
- Aufzählungskonstante haben einen ganzzahligen Wert (beginnend mit 0) und sind von Datentyp int
- Aufzählungstypen sind geeignet um Konstanten zu definieren und können ihre Werte auch direkt zugewiesen bekommen

```
enum Color { RED=1, GREEN=3, BLUE=5, YELLOW=7 }
```

- Aber, nicht schön: korrekte Zuweisung an Variablen von Aufzählungstypen wird in C nicht geprüft

```
c = 9; // obwohl eigentlich ein Fehler, in C eine korrekte Zuweisung
```

Ein- und Ausgabe von Werten

Ein- und Ausgabefunktionen sind in <stdio.h> beschrieben

Eingabe von der Tastatur

```
int x;
scanf("%d", &x);        // liest eine Zahl vom Eingabestrom
```

Ausgabe auf dem Bildschirm

```
int x = 12;
printf("Integer Zahl: %d ", x);      // Ausgabe einer Zeichenkette
printf("Integer Zahl: %d \n", x);    // diesmal mit newline
```

Umwandlungszeichen

Zeichen	Typ	Ausgabe erfolgt
%d, %i	int	dezimal mit Vorzeichen
%o	unsigned int	oktal ohne Vorzeichen
%x,%X	unsigned int	hexadezimal ohne Vorzeichen
%u	unsigned int	ohne Vorzeichen in dezimaler Form
%c	int	als Zeichen

Grundstruktur von C-Programmen

- Erstellen der Datei ***programName***.c

```
/* Datei: programName.c */
// Präprozessoranweisungen
#include <stdio.h>
                …
#define MAX 100
                ...
// Funktionsdeklarationen
                ...
int main(void){
        // Deklarationen
        // Anweisungen
}
```

programName frei wählbar

weitere wichtige Optionen:
- std=XXX, legt Sprachstandard fest
- Wall und -Wextra, aktiviert zusätzliche Warnungen
- pedantic-errors, erzwingt Fehlerausgaben für nicht iso-konforme Verwendungen

- Übersetzen des Quelltextes:

```
gcc -o programName programName.c
```

- Ausführen des Programmes:

```
programName
```

Beispiel eines C-Programms

- Erstellen der Datei sample.c

```
/* Datei: sample.c */
#include <stdio.h>
int main(void){
    int a, b, sum;
        printf("Geben Sie 2 Zahlen ein:");
        scanf("%d %d", &a, &b);
        sum = a + b;
        printf("Summe = %d  \n", sum);
        return 0;
}
```

- Übersetzen des Quelltextes

```
gcc -o sample sample.c          erzeugt Objektdatei sample
```

- Ausführen des Programmes

```
sample                          ruft main-Methode auf
Geben Sie 2 Zahlen ein: 3  4    Eingabe mit Return-Taste abschließen
Summe = 7
```